

TPN3.2 - PROGRAMACIÓN DE SOCKETS

Ejercicio 1)

1. Chat mediante Sockets

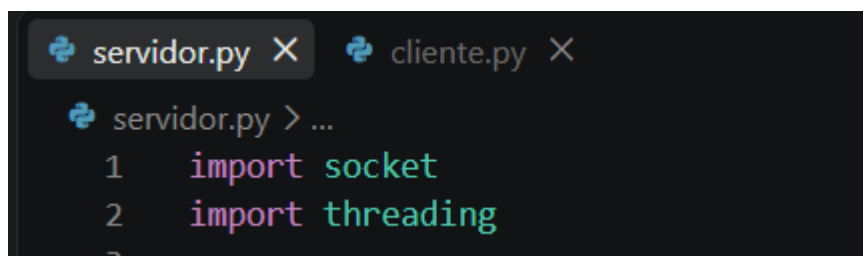
Para el desarrollo del trabajo práctico se implementó una aplicación de chat utilizando el lenguaje de programación Python y la arquitectura cliente-servidor.

La aplicación está compuesta por dos programas principales: [servidor.py](#) , [cliente.py](#)

El archivo “[servidor.py](#)” se encarga de recibir las conexiones de los clientes, mantener la comunicación con ellos y retransmitir los mensajes hacia los demás usuarios conectados.

El archivo “cliente.py” permite que cada usuario se conecte al servidor, ingrese un nombre de usuario, envíe mensajes y reciba en tiempo real los mensajes enviados por otros clientes.

La comunicación se realiza mediante sockets TCP/IP. En Python se utilizan las librerías:

A screenshot of a code editor with a dark background. At the top, there are two tabs: 'servidor.py' and 'cliente.py', both with a blue icon and a close button. The 'servidor.py' tab is active, showing the first two lines of code: '1 import socket' and '2 import threading'. The code is color-coded: 'import' is in pink, 'socket' and 'threading' are in green, and the line numbers are in light blue. The cursor is at the end of the second line.

```
servidor.py > ...  
1  import socket  
2  import threading
```

La librería “socket” permite crear y utilizar los sockets necesarios para establecer la comunicación entre el cliente y el servidor.

La librería “threading” permite manejar múltiples clientes de manera concurrente, de forma que el servidor pueda atender varias conexiones al mismo tiempo.

- CONFIGURACIÓN UTILIZADA

Para el servidor se definió el puerto y dirección:

```
3
4  HOST = "0.0.0.0"
5  PORT = 5000
6
```

El valor "0.0.0.0" permite que el servidor quede a la escucha en todas las interfaces de red disponibles del equipo.

En el cliente, para las primeras pruebas realizadas en el mismo equipo, se utilizó:

```
HOST = "127.0.0.1"
PORT = 5000
```

La dirección "127.0.0.1" corresponde a la interfaz de loopback o localhost, permitiendo realizar la comunicación entre cliente y servidor dentro de la misma computadora.

- CREACIÓN DEL SOCKET DEL SERVIDOR

El socket del servidor se crea mediante:

```
def iniciar_servidor():
    servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Donde, "AF_INET" indica el uso del protocolo IPv4. Mientras que "SOCK_STREAM" indica que se utiliza un socket orientado a conexión mediante el protocolo TCP.

Luego el servidor asocia el socket con la dirección IP y el puerto mediante:

```
try:
    servidor.bind((HOST, PORT))
```

Después se coloca el servidor en modo de escucha:

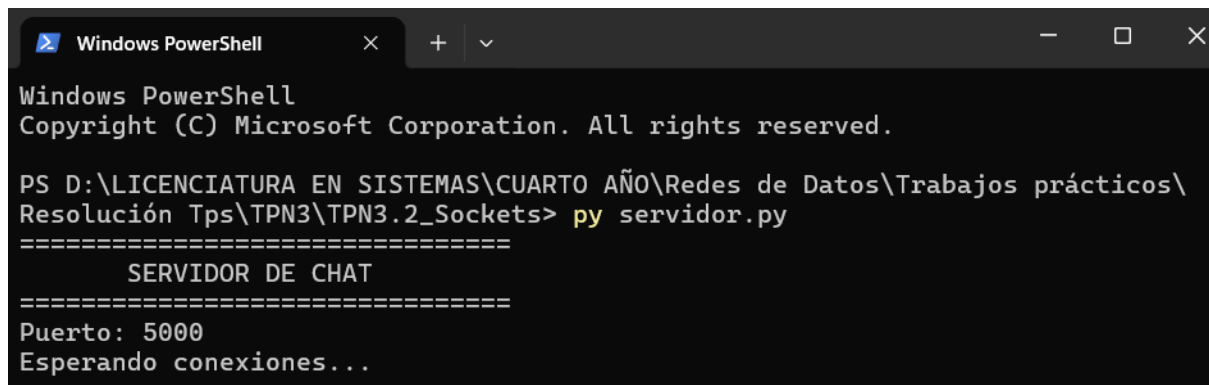
```
servidor.listen()
```

Finalmente, el servidor queda esperando conexiones de clientes mediante:

```
while True:
    cliente, direccion = servidor.accept()
```

Cuando un cliente se conecta, el servidor obtiene un nuevo socket para comunicarse específicamente con ese cliente y también obtiene su dirección IP y puerto de origen.

SERVIDOR DE CHAT INICIADO Y ESCUCHANDO CONEXIONES



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\LICENCIATURA EN SISTEMAS\CUARTO AÑO\Redes de Datos\Trabajos prácticos\
Resolución Tps\TPN3\TPN3.2_Sockets> py servidor.py
=====
SERVIDOR DE CHAT
=====
Puerto: 5000
Esperando conexiones...
```

En la captura se observa la ejecución del servidor desarrollado en Python. El servidor queda a la escucha en el puerto TCP 5000 esperando conexiones provenientes de los clientes.

2. Comunicación entre dos o más usuarios

Una vez iniciado el servidor, se ejecutaron varias instancias del programa “cliente.py” para comprobar la comunicación simultánea entre diferentes usuarios.

Cada cliente establece una conexión TCP con el servidor mediante:

```
cliente.connect((HOST, PORT))
```

Para las pruebas realizadas en el mismo equipo se utilizó:

```
HOST = "127.0.0.1"
PORT = 5000
```

Una vez conectado, cada cliente ingresa su nombre de usuario y lo envía al servidor mediante:

```
cliente.send(nombre.encode("utf-8"))
```

El servidor recibe este dato con:

```
nombre = cliente.recv(1024).decode("utf-8").strip()
```

De esta forma, el servidor puede identificar a cada usuario conectado.

- Conexión de varios clientes

Para verificar el funcionamiento se ejecutaron tres clientes diferentes, por ejemplo: Leo, Leopoldo y Francisco.

Cada cliente establece una conexión independiente con el servidor.

El servidor muestra cada nueva conexión mediante mensajes similares a:

```
[NUEVO CLIENTE] Hola - ('127.0.0.1', 54244)
[Hola] Leo
[DESCONECTADO] Hola
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Leo - ('127.0.0.1', 54249)
[Leo] Holaaa
[CONEXIONES ACTIVAS] 2
[NUEVO CLIENTE] Francisco - ('127.0.0.1', 58336)
[CONEXIONES ACTIVAS] 3
[NUEVO CLIENTE] Leopoldo - ('127.0.0.1', 58339)
[Leopoldo] Hola a todos
[Francisco] Hola Leopoldo, me aprobas?
[Leo] Hola, yo también me llamo leo
```

En la captura se observan a los tres clientes conectados simultáneamente al servidor. Cada conexión posee su propia dirección IP y puerto de origen, mientras que todas utilizan como destino el puerto TCP 5000 del servidor.

- Envió de mensajes

Cuando un usuario escribe un mensaje, el cliente lo envía al servidor mediante:

```
cliente.send(nombre.encode("utf-8"))
```

El servidor recibe el mensaje mediante:

```
mensaje = cliente.recv(1024).decode("utf-8")
```

Posteriormente, el servidor identifica al usuario que envió el mensaje y genera:

```
mensaje_completo = f"[{nombre}] {mensaje}"
```

Luego utiliza la función:

```
def enviar_a_todos(mensaje, cliente_origen=None):
    for cliente in clientes.copy():
        if cliente != cliente_origen:
            try:
                cliente.send(mensaje.encode("utf-8"))
            except:
                eliminar_cliente(cliente)
```

para transmitir el mensaje a los demás clientes conectados.

Ejemplo comunicación entre clientes:

The image displays two side-by-side Windows PowerShell windows. The left window shows the server's log output, including connection status, client names, and IP addresses. The right window shows the client's perspective, including prompts for username and messages received from the server and other clients.

```

[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Leo - ('127.0.0.1', 52078)
[Leo] Hola servidor
[DESCONECTADO] Leo
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Hola - ('127.0.0.1', 54244)
[Hola] Leo
[DESCONECTADO] Hola
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Leo - ('127.0.0.1', 54249)
[Leo] Holaaa
[CONEXIONES ACTIVAS] 2
[NUEVO CLIENTE] Francisco - ('127.0.0.1', 58336)
[CONEXIONES ACTIVAS] 3
[NUEVO CLIENTE] Leopoldo - ('127.0.0.1', 58339)
[Leopoldo] Hola a todos
[Francisco] Hola Leopoldo, me approves?
[Leo] Hola, yo también me llamo leo
[DESCONECTADO] Leo
[CONEXIONES ACTIVAS] 3
[NUEVO CLIENTE] Tumacho - ('127.0.0.1', 55169)
[DESCONECTADO] Tumacho
  
```

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\LICENCIATURA EN SISTEMAS\CUARTO AÑO\Redes de Datos\Trabajos prácticos\Resolución Tps\TPN3\TPN3.2_Sockets> python cliente.py
=====
CLIENTE CHAT
=====
Ingrese su nombre de usuario: Francisco

[SERVIDOR] Conexión establecida correctamente.
>
[SERVIDOR] Leopoldo ingresó al chat.
>
[Leopoldo] Hola a todos
> Hola Leopoldo, me approves?
>
[Leo] Hola, yo también me llamo leo
>
[SERVIDOR] Leo se desconectó del chat.

=====
CLIENTE CHAT
=====
Ingrese su nombre de usuario: Leopoldo

[SERVIDOR] Conexión establecida correctamente.
>
> Hola a todos
[Francisco] Hola Leopoldo, me approves?
>
[Leo] Hola, yo también me llamo leo
> /listar

[SERVIDOR] Usuarios conectados: Leo, Francisco, Leopoldo
>
[SERVIDOR] Leo se desconectó del chat.
>
[SERVIDOR] Tumacho ingresó al chat.
>
[SERVIDOR] Tumacho se desconectó del chat.
>
  
```

En la captura se observa el envío de un mensaje desde el cliente identificado como Leopoldo y su recepción en los demás clientes conectados. El servidor funciona como intermediario, recibiendo el mensaje y transmitiendo hacia los otros usuarios.

- Concurrencia de clientes

Para permitir que varios usuarios puedan utilizar el chat simultáneamente se utilizó la librería:

```
import threading
```

Cada vez que el servidor acepta una nueva conexión:

```
cliente, direccion = servidor.accept()
```

Se crea un hilo independiente:

```
hilo = threading.Thread(  
    target=manejar_cliente,  
    args=(cliente, direccion),  
    daemon=True  
)  
  
hilo.start()
```

De esta manera, cada cliente puede enviar y recibir información sin bloquear las conexiones de los demás usuarios.

3. Interfaz de usuarios del cliente

La aplicación cliente fue desarrollada con una interfaz de consola sencilla utilizando Python. Al iniciar el programa, el usuario visualiza un encabezado y posteriormente se solicita el ingreso de su nombre.

El código utilizado es:

```
print("=====  
print("      CLIENTE CHAT")  
print("=====  
  
nombre = input("Ingrese su nombre de usuario: ")
```

Esto permite identificar a cada participante dentro del chat.

Una vez ingresado el nombre, el cliente lo envía al servidor mediante:

```
cliente.send(nombre.encode("utf-8"))
```

El uso de: “encode(“utf-8”), convierte el texto ingresado por el usuario a una secuencia de bytes, ya que los sockets transmiten datos en este formato.

Ingreso del nombre de usuario

```
=====
                CLIENTE CHAT
=====
Ingrese su nombre de usuario: Leopoldo
```

En la captura se observa la interfaz inicial del cliente, donde el usuario ingresa su nombre antes de comenzar a utilizar el chat. Este dato es enviado posteriormente al servidor para identificar al participante durante la comunicación.

- Envío de mensajes

Luego de conectarse, el programa entra en un ciclo donde espera que el usuario escriba mensajes:

```
while True:
    mensaje = input("> ")

    cliente.send(mensaje.encode("utf-8"))

    if mensaje == "/quitar":
        break
```

El método “input("> ")” permite ingresar mensajes desde la consola.

- Recepción de mensajes en tiempo real

Uno de los requisitos importantes es que un cliente pueda recibir mensajes sin tener que enviar primero uno propio.

Para lograrlo se utiliza un hilo independiente:

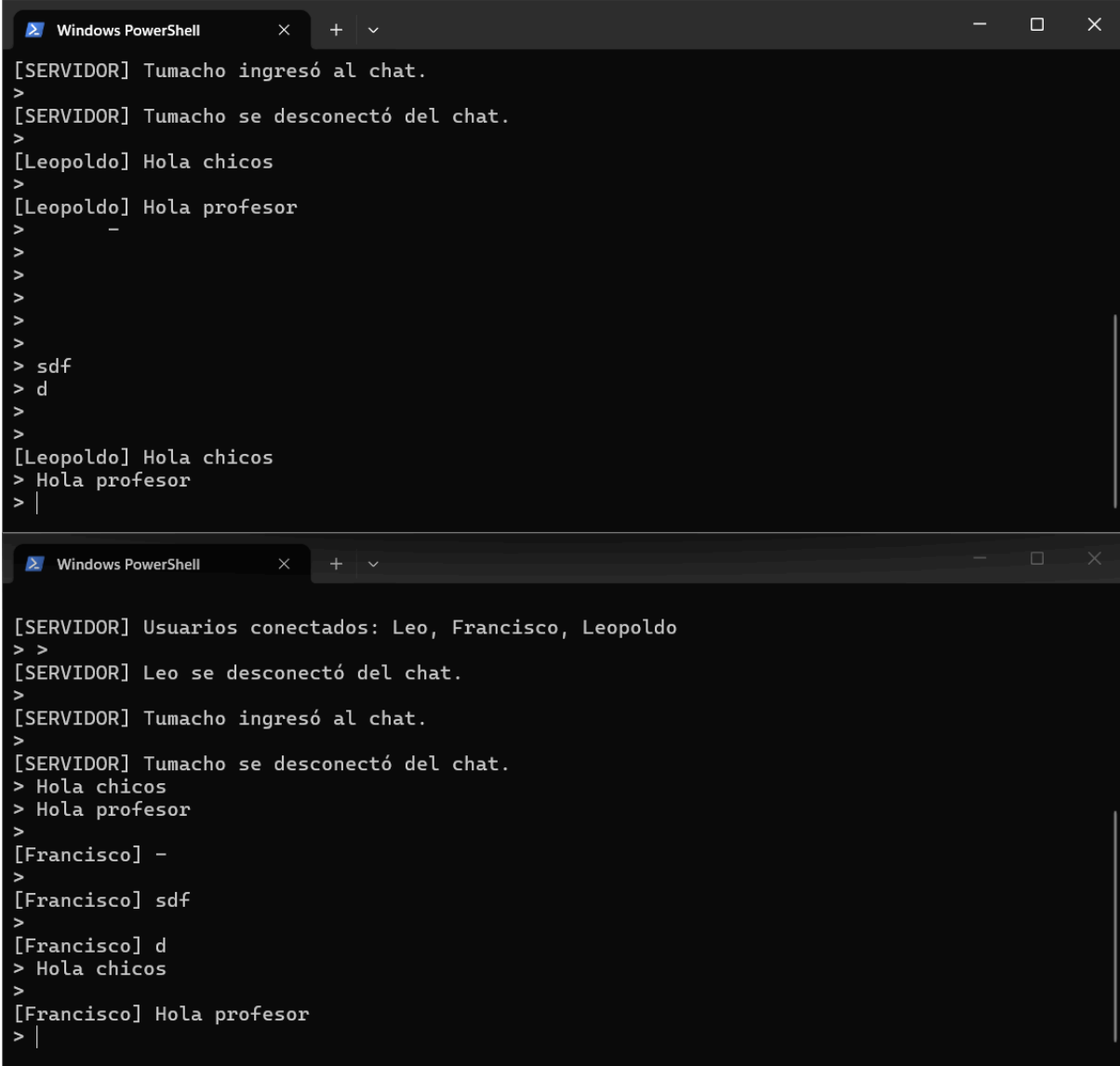
```
hilo = threading.Thread(  
    target=recibir_mensajes,  
    args=(cliente,),  
    daemon=True  
)  
  
hilo.start()
```

Este hilo ejecuta la función:

```
def recibir_mensajes(cliente):  
    while True:  
        try:  
            mensaje = cliente.recv(1024).decode("utf-8")  
  
            if not mensaje:  
                print("\n[INFO] El servidor cerró la conexión.")  
                break  
  
            print(f"\n{mensaje}")  
            print("> ", end="", flush=True)  
  
        except:  
            print("\n[INFO] Se perdió la conexión con el servidor.")  
            break
```

La instrucción principal es: "cliente.recv(1024)" que permite recibir datos provenientes del servidor. Posteriormente: ".decode("utf-8")" convierte los bytes recibidos nuevamente en texto para poder mostrarlos en pantalla.

Envío y recepción de mensajes en tiempo real



```
Windows PowerShell
[SERVIDOR] Tumacho ingresó al chat.
>
[SERVIDOR] Tumacho se desconectó del chat.
>
[Leopoldo] Hola chicos
>
[Leopoldo] Hola profesor
>
-
>
>
>
>
>
>
>
> sdf
> d
>
>
[Leopoldo] Hola chicos
> Hola profesor
> |

Windows PowerShell
[SERVIDOR] Usuarios conectados: Leo, Francisco, Leopoldo
> >
[SERVIDOR] Leo se desconectó del chat.
>
[SERVIDOR] Tumacho ingresó al chat.
>
[SERVIDOR] Tumacho se desconectó del chat.
>
> Hola chicos
> Hola profesor
>
[Francisco] -
>
[Francisco] sdf
>
[Francisco] d
> Hola chicos
>
[Francisco] Hola profesor
> |
```

En la captura se observa el intercambio de mensajes entre dos clientes. Cada usuario puede escribir mensajes y visualizar en tiempo real los mensajes recibidos desde otros clientes a través del servidor.

4. Comunicación entre clientes y servidor central mediante sockets

La aplicación utiliza una arquitectura “Cliente-Servidor”, donde todos los clientes se conectan a un servidor central.

El servidor actúa como intermediario entre los usuarios: recibe los mensajes de un cliente y los envía a los demás clientes conectados.

- Comunicación mediante un servidor central

En esta actividad se verificó que los clientes establecen una conexión con un servidor central, el cual actúa como intermediario de la comunicación.

El servidor queda a la espera de nuevas conexiones mediante:

```
cliente, direccion = servidor.accept()
```

Cada vez que un cliente se conecta, el servidor obtiene un socket exclusivo para comunicarse con ese usuario y registra su dirección IP y puerto de origen.

Posteriormente, cuando un cliente envía un mensaje, el servidor lo recibe y lo identifica con el nombre del usuario:

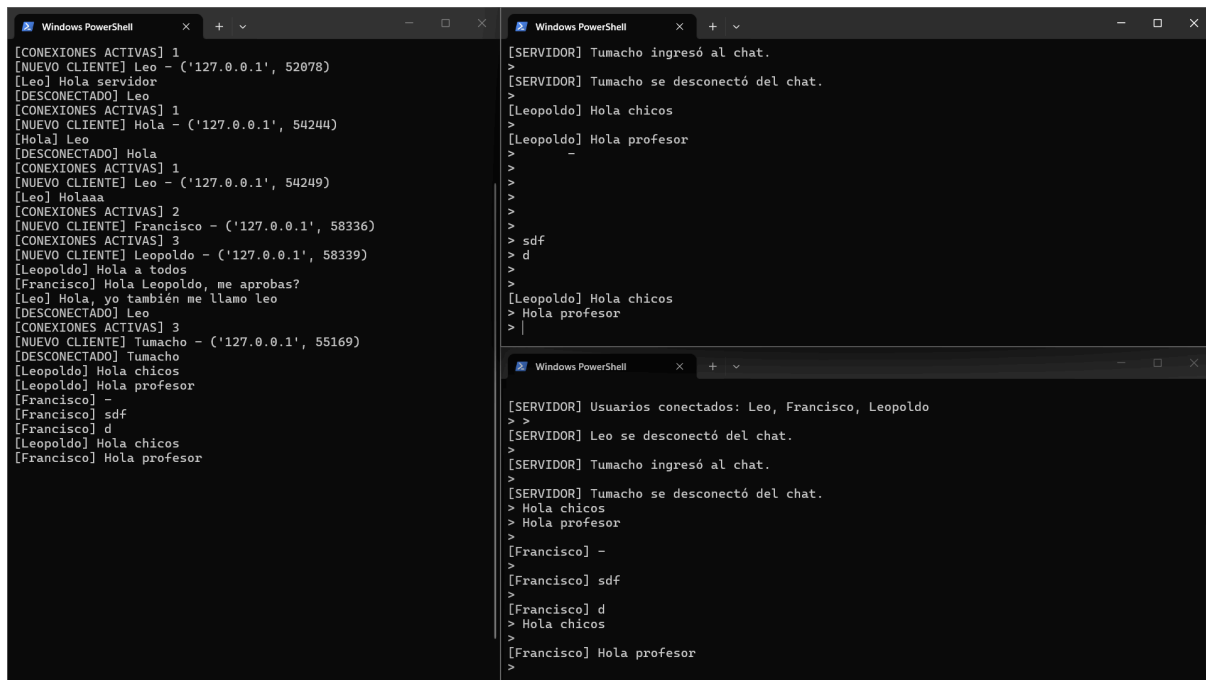
```
mensaje_completo = f"[{nombre}] {mensaje}"
```

Luego utiliza la función:

```
def enviar_a_todos(mensaje, cliente_origen=None):  
    for cliente in clientes.copy():  
        if cliente != cliente_origen:  
            try:  
                cliente.send(mensaje.encode("utf-8"))  
            except:  
                eliminar_cliente(cliente)
```

Para retransmitir el contenido hacia los demás clientes conectados.

Retransmisión de mensajes mediante el servidor



```
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Leo - ('127.0.0.1', 52078)
[Leo] Hola servidor
[DESCONECTADO] Leo
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Hola - ('127.0.0.1', 54244)
[Hola] Leo
[DESCONECTADO] Hola
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Leo - ('127.0.0.1', 54249)
[Leo] Holaaa
[CONEXIONES ACTIVAS] 2
[NUEVO CLIENTE] Francisco - ('127.0.0.1', 58336)
[CONEXIONES ACTIVAS] 3
[NUEVO CLIENTE] Leopoldo - ('127.0.0.1', 58339)
[Leopoldo] Hola a todos
[Francisco] Hola Leopoldo, me aprobas?
[Leo] Hola, yo también me llamo leo
[DESCONECTADO] Leo
[CONEXIONES ACTIVAS] 3
[NUEVO CLIENTE] Tumacho - ('127.0.0.1', 55169)
[DESCONECTADO] Tumacho
[Leopoldo] Hola chicos
[Leopoldo] Hola profesor
[Francisco] -
[Francisco] sdf
[Francisco] d
[Leopoldo] Hola chicos
[Francisco] Hola profesor

[SERVIDOR] Tumacho ingresó al chat.
>
[SERVIDOR] Tumacho se desconectó del chat.
>
[Leopoldo] Hola chicos
>
[Leopoldo] Hola profesor
>
>
>
> sdf
> d
>
>
[Leopoldo] Hola chicos
> Hola profesor
>

[SERVIDOR] Usuarios conectados: Leo, Francisco, Leopoldo
>
[SERVIDOR] Leo se desconectó del chat.
>
[SERVIDOR] Tumacho ingresó al chat.
>
[SERVIDOR] Tumacho se desconectó del chat.
> Hola chicos
> Hola profesor
>
[Francisco] -
>
[Francisco] sdf
>
[Francisco] d
> Hola chicos
>
[Francisco] Hola profesor
>
```

En la captura se observa el funcionamiento del servidor central. El mensaje enviado por un cliente es recibido por el servidor y posteriormente retransmitido hacia los demás usuarios conectados.

5. Manejo de Errores y desconexiones

En esta actividad se implementó el control de desconexiones para evitar que la salida de un cliente provoque la caída del servidor o afecte a los demás usuarios conectados.

Cuando un cliente deja de enviar datos o cierra la conexión, el servidor detecta la situación y finaliza únicamente la comunicación con ese usuario.

La eliminación del cliente se realiza mediante:

```
def eliminar_cliente(cliente):
    if cliente in clientes:
        clientes.remove(cliente)

    nombre = usuarios.pop(cliente, "Usuario desconocido")

    try:
        cliente.close()
    except:
        pass

    print(f"[DESCONECTADO] {nombre}")

    enviar_a_todos(
        f"[SERVIDOR] {nombre} se desconectó del chat."
    )
```

Esta función realiza tres tareas principales: Elimina al cliente de la lista de conexiones activas, cierra un socket e informa a los demás usuarios que el cliente se desconectó.

- Detección de una desconexión

Dentro de la función que atiende a cada cliente se utiliza:

```
mensaje = cliente.recv(1024).decode("utf-8")

if not mensaje:
    break
```

Si “recv()” no obtiene datos, se interpreta que el cliente cerró su conexión.

Al salir del ciclo se ejecuta:

```
finally:
    eliminar_cliente(cliente)
```

De esta manera, incluso si ocurre un error, se intenta limpiar correctamente la conexión del usuario.

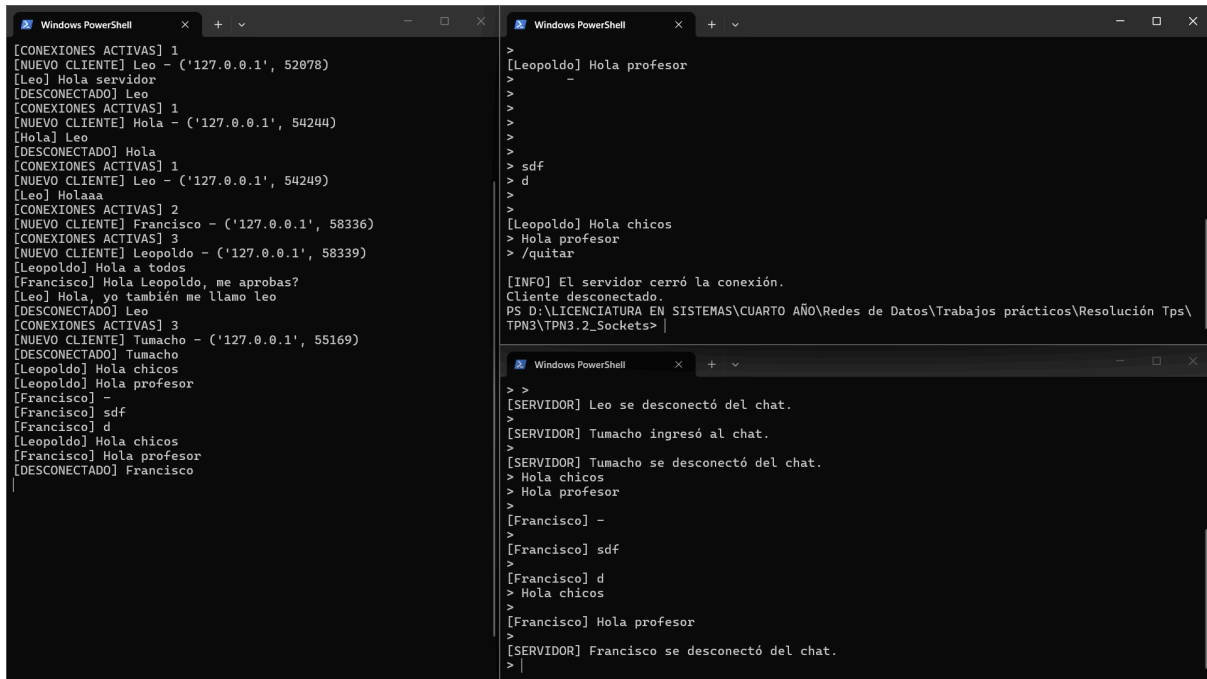
- Manejo de desconexiones inesperadas

También se contemplan situaciones en las que un cliente cierre abruptamente su aplicación o pierda la conexión:

```
except (ConnectionResetError, ConnectionAbortedError):
    pass
```

Esto evita que una desconexión inesperada haga finalizar el servidor completo.

Desconexión controlada de un cliente



```
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Leo - ('127.0.0.1', 52078)
[Leo] Hola servidor
[DESCONECTADO] Leo
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Hola - ('127.0.0.1', 54244)
[Hola] Leo
[DESCONECTADO] Hola
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Leo - ('127.0.0.1', 54249)
[Leo] Holaaa
[CONEXIONES ACTIVAS] 2
[NUEVO CLIENTE] Francisco - ('127.0.0.1', 58336)
[CONEXIONES ACTIVAS] 3
[NUEVO CLIENTE] Leopoldo - ('127.0.0.1', 58339)
[Leopoldo] Hola a todos
[Francisco] Hola Leopoldo, me aprobas?
[Leo] Hola, yo también me llamo leo
[DESCONECTADO] Leo
[CONEXIONES ACTIVAS] 3
[NUEVO CLIENTE] Tumacho - ('127.0.0.1', 55169)
[DESCONECTADO] Tumacho
[Leopoldo] Hola chicos
[Leopoldo] Hola profesor
[Francisco] -
[Francisco] sdf
[Francisco] d
[Leopoldo] Hola chicos
[Francisco] Hola profesor
[DESCONECTADO] Francisco

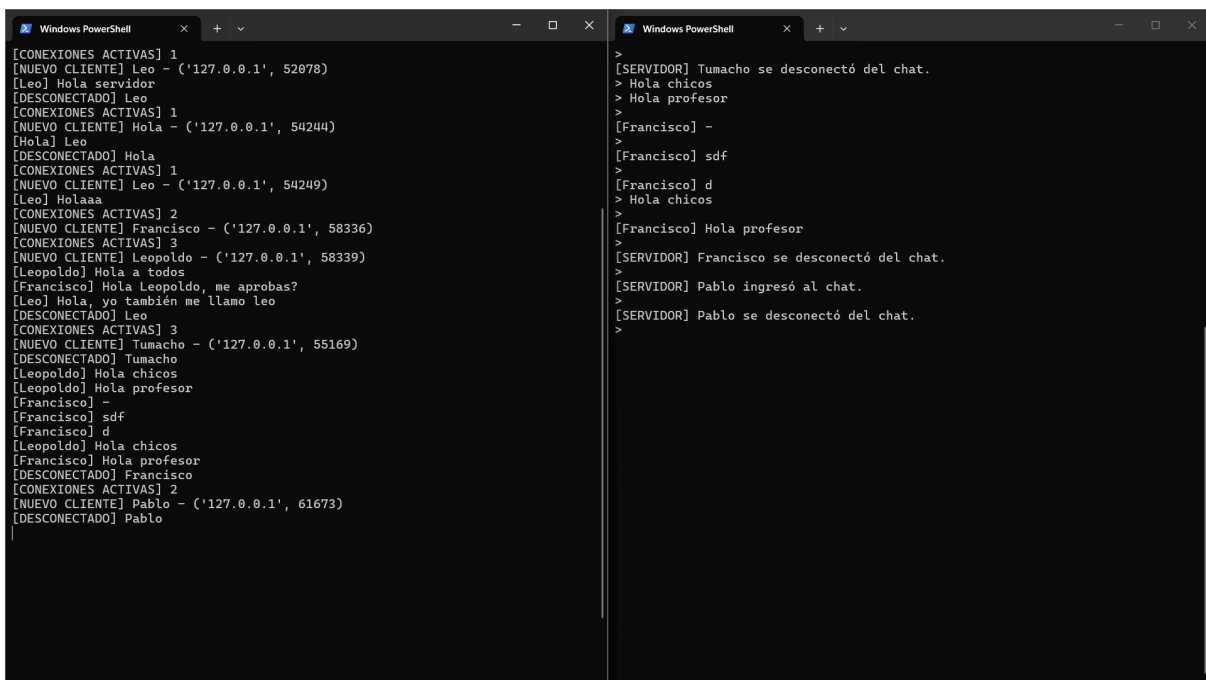
>
[Leopoldo] Hola profesor
> -
>
>
>
> sdf
> d
>
[Leopoldo] Hola chicos
> Hola profesor
> /quitar

[INFO] El servidor cerró la conexión.
Cliente desconectado.
PS D:\LICENCIATURA EN SISTEMAS\CUARTO AÑO\Redes de Datos\Trabajos prácticos\Resolución Tps\
TPN3\TPN3.2_Sockets>
```

```
>>
[SERVIDOR] Leo se desconectó del chat.
>
[SERVIDOR] Tumacho ingresó al chat.
>
[SERVIDOR] Tumacho se desconectó del chat.
> Hola chicos
> Hola profesor
>
[Francisco] -
>
[Francisco] sdf
>
[Francisco] d
> Hola chicos
>
[Francisco] Hola profesor
>
[SERVIDOR] Francisco se desconectó del chat.
> |
```

Se observa una desconexión controlada mediante el comando “/quitar”. El servidor elimina al usuario desconectado, cierra su socket y notifica la salida al resto de los participantes sin interrumpir el funcionamiento del chat.

Desconexión inesperada sin caída del servidor



```
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Leo - ('127.0.0.1', 52078)
[Leo] Hola servidor
[DESCONECTADO] Leo
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Hola - ('127.0.0.1', 54244)
[Hola] Leo
[DESCONECTADO] Hola
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Leo - ('127.0.0.1', 54249)
[Leo] Holaaa
[CONEXIONES ACTIVAS] 2
[NUEVO CLIENTE] Francisco - ('127.0.0.1', 58336)
[CONEXIONES ACTIVAS] 3
[NUEVO CLIENTE] Leopoldo - ('127.0.0.1', 58339)
[Leopoldo] Hola a todos
[Francisco] Hola Leopoldo, me aprobas?
[Leo] Hola, yo también me llamo leo
[DESCONECTADO] Leo
[CONEXIONES ACTIVAS] 3
[NUEVO CLIENTE] Tumacho - ('127.0.0.1', 55169)
[DESCONECTADO] Tumacho
[Leopoldo] Hola chicos
[Leopoldo] Hola profesor
[Francisco] -
[Francisco] sdf
[Francisco] d
[Leopoldo] Hola chicos
[Francisco] Hola profesor
[DESCONECTADO] Francisco
[CONEXIONES ACTIVAS] 2
[NUEVO CLIENTE] Pablo - ('127.0.0.1', 61673)
[DESCONECTADO] Pablo

>
[SERVIDOR] Tumacho se desconectó del chat.
> Hola chicos
> Hola profesor
>
[Francisco] -
>
[Francisco] sdf
>
[Francisco] d
> Hola chicos
>
[Francisco] Hola profesor
>
[SERVIDOR] Francisco se desconectó del chat.
>
[SERVIDOR] Pablo ingresó al chat.
>
[SERVIDOR] Pablo se desconectó del chat.
>
```

Se realizó una desconexión abrupta cerrando directamente uno de los clientes. El servidor detectó la pérdida de conexión, eliminó al usuario correspondiente y continuó atendiendo normalmente a los demás clientes conectados.

6. Implementación de comandos especiales

El comando “/listar” permite consultar qué usuarios se encuentran conectados en ese momento.

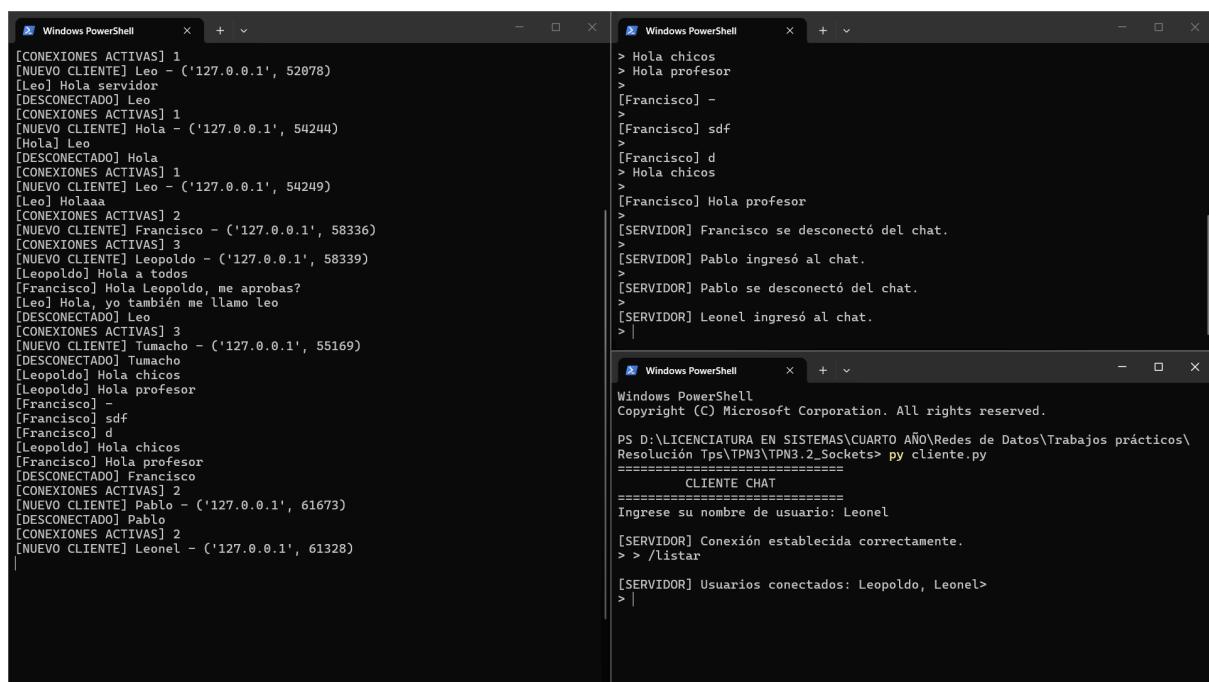
En el servidor se controla mediante:

```
if mensaje == "/listar":
    lista = ", ".join(usuarios.values())

    cliente.send(
        f"[SERVIDOR] Usuarios conectados: {lista}".encode("utf-8")
    )
```

Cuando el servidor recibe “/listar”, obtiene los nombres almacenados en usuarios y genera una respuesta dirigida únicamente al cliente que realizó la consulta.

Funcionamiento del comando /listar



The screenshot displays three Windows PowerShell windows. The leftmost window shows a log of server activity, including connection status changes (e.g., [CONEXIONES ACTIVAS] 1, [NUEVO CLIENTE] Leo) and chat messages (e.g., [Leo] Hola servidor, [Francisco] Hola chicos). The middle window shows a client's perspective, with messages like '> Hola chicos', '> Hola profesor', and '[Francisco] sdf'. The rightmost window shows the server's response to the '/listar' command: '[SERVIDOR] Usuarios conectados: Leopoldo, Leonel'.

Se observa el funcionamiento del comando /listar, mediante el cual un cliente solicita al servidor la lista de usuarios conectados actualmente al chat.

El comando “/quitar” permite que un cliente finalice voluntariamente su conexión, en el servidor se detecta mediante:

```
elif mensaje == "/quitar":  
    break
```

Al salir del ciclo, se ejecuta el procedimiento de eliminación del cliente y cierre del socket.

En el programa cliente también se comprueba el comando:

```
if mensaje == "/quitar":  
    break
```

Después se ejecuta:

```
finally:  
    cliente.close()  
    print("Cliente desconectado.")
```

De esta forma, la desconexión se realiza de manera controlada.

Funcionamiento del comando /quitar

```
Windows PowerShell
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Leo - ('127.0.0.1', 52078)
[Leo] Hola servidor
[DESCONECTADO] Leo
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Hola - ('127.0.0.1', 54244)
[Hola] Leo
[DESCONECTADO] Hola
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Leo - ('127.0.0.1', 54249)
[Leo] Holaaa
[CONEXIONES ACTIVAS] 2
[NUEVO CLIENTE] Francisco - ('127.0.0.1', 58336)
[CONEXIONES ACTIVAS] 3
[NUEVO CLIENTE] Leopoldo - ('127.0.0.1', 58339)
[Leopoldo] Hola a todos
[Francisco] Hola Leopoldo, me approves?
[Leo] Hola, yo también me llamo Leo
[DESCONECTADO] Leo
[CONEXIONES ACTIVAS] 3
[NUEVO CLIENTE] Tumacho - ('127.0.0.1', 55169)
[DESCONECTADO] Tumacho
[Leopoldo] Hola chicos
[Leopoldo] Hola profesor
[Francisco] sdf
[Francisco] d
[Leopoldo] Hola chicos
[Francisco] Hola profesor
[DESCONECTADO] Francisco
[CONEXIONES ACTIVAS] 2
[NUEVO CLIENTE] Pablo - ('127.0.0.1', 61673)
[DESCONECTADO] Pablo
[CONEXIONES ACTIVAS] 2
[NUEVO CLIENTE] Leonel - ('127.0.0.1', 61328)
[DESCONECTADO] Leopoldo

Windows PowerShell
[Francisco] sdf
>
[Francisco] d
> Hola chicos
>
[Francisco] Hola profesor
>
[SERVIDOR] Francisco se desconectó del chat.
>
[SERVIDOR] Pablo ingresó al chat.
>
[SERVIDOR] Pablo se desconectó del chat.
>
[SERVIDOR] Leonel ingresó al chat.
> /quitar
[INFO] El servidor cerró la conexión.
Cliente desconectado.
PS D:\LICENCIATURA EN SISTEMAS\CUARTO AÑO\Redes de Datos\Trabajos prácticos\Resolución Tps\TPN3\TPN3.2_Sockets>

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\LICENCIATURA EN SISTEMAS\CUARTO AÑO\Redes de Datos\Trabajos prácticos\Resolución Tps\TPN3\TPN3.2_Sockets> py cliente.py
=====
CLIENTE CHAT
=====
Ingrese su nombre de usuario: Leonel

[SERVIDOR] Conexión establecida correctamente.
> > /listar
[SERVIDOR] Usuarios conectados: Leopoldo, Leonel>
>
[SERVIDOR] Leopoldo se desconectó del chat.
>|
```

Se verifica el funcionamiento del comando “/quitar”, utilizado para finalizar voluntariamente la conexión de un cliente con el servidor.

7. Informe explicativo del código

A lo largo de las actividades anteriores se documentaron las principales líneas de código utilizadas para establecer y mantener la comunicación mediante sockets entre el cliente y el servidor.

La implementación utiliza las librerías:

```
servidor.py > ...  
1  import socket  
2  import threading  
3
```

La librería “Socket” permite establecer la comunicación TCP/IP, mientras que el “Threading” permite atender múltiples clientes de forma concurrente.

Las principales operaciones utilizadas fueron:

```
servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

para crear sockets TCP sobre IPv4.

```
try:  
    servidor.bind((HOST, PORT))  
    servidor.listen()  
  
    print("=====  
    print("      SERVIDOR DE CHAT")  
    print("=====  
    print(f"Puerto: {PORT}")  
    print("Esperando conexiones...")  
    print()  
  
    while True:  
        cliente, direccion = servidor.accept()
```

para configurar el servidor y aceptar conexiones.


```
cliente.send(  
    f"[SERVIDOR] Usuarios conectados: {lista}".encode("utf-8")  
)
```

```
mensaje = cliente.recv(1024).decode("utf-8")
```

Para el intercambio de datos.

8. Código fuente del servidor y cliente, pantalla del chat en ejecución, interacción entre diferentes clientes.

Servidor.py:

```
import socket  
import threading  
  
HOST = "0.0.0.0"  
PORT = 5000  
  
clientes = []  
usuarios = {}  
  
def enviar_a_todos(mensaje, cliente_origen=None):  
    for cliente in clientes.copy():  
        if cliente != cliente_origen:  
            try:  
                cliente.send(mensaje.encode("utf-8"))  
            except:  
                eliminar_cliente(cliente)  
  
def eliminar_cliente(cliente):  
    if cliente in clientes:  
        clientes.remove(cliente)  
  
    nombre = usuarios.pop(cliente, "Usuario desconocido")  
  
    try:  
        cliente.close()  
    except:
```

```
pass

print(f"[DESCONECTADO] {nombre}")

enviar_a_todos(
    f"[SERVIDOR] {nombre} se desconectó del chat."
)

def manejar_cliente(cliente, direccion):
    try:
        nombre = cliente.recv(1024).decode("utf-8").strip()

        if not nombre:
            cliente.close()
            return

        clientes.append(cliente)
        usuarios[cliente] = nombre

        print(f"[NUEVO CLIENTE] {nombre} - {direccion}")

        cliente.send(
            "[SERVIDOR] Conexión establecida
correctamente.".encode("utf-8")
        )

        enviar_a_todos(
            f"[SERVIDOR] {nombre} ingresó al chat.",
            cliente
        )

        while True:
            mensaje = cliente.recv(1024).decode("utf-8")

            if not mensaje:
                break

            mensaje = mensaje.strip()

            if mensaje == "/listar":
                lista = ", ".join(usuarios.values())
```

```

        cliente.send(
            f"[SERVIDOR] Usuarios conectados:
{lista}".encode("utf-8")
        )

    elif mensaje == "/quitar":
        break

    else:
        mensaje_completo = f"[{nombre}] {mensaje}"

        print(mensaje_completo)

        enviar_a_todos(
            mensaje_completo,
            cliente
        )

except (ConnectionResetError, ConnectionAbortedError):
    pass

except Exception as error:
    print(f"[ERROR] Cliente {direccion}: {error}")

finally:
    eliminar_cliente(cliente)

def iniciar_servidor():
    servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    try:
        servidor.bind((HOST, PORT))
        servidor.listen()

        print("=====")
        print("      SERVIDOR DE CHAT")
        print("=====")
        print(f"Puerto: {PORT}")
        print("Esperando conexiones...")
        print()

        while True:

```

```

        cliente, direccion = servidor.accept()

        hilo = threading.Thread(
            target=manejar_cliente,
            args=(cliente, direccion),
            daemon=True
        )

        hilo.start()

        print(
            f"[CONEXIONES ACTIVAS] {threading.active_count() - 1}"
        )

    except OSError as error:
        print(f"[ERROR DEL SERVIDOR] {error}")

    finally:
        servidor.close()

if __name__ == "__main__":
    iniciar_servidor()

```

Cliente.py:

```

import socket
import threading

HOST = "127.0.0.1"
PORT = 5000

def recibir_mensajes(cliente):
    while True:
        try:
            mensaje = cliente.recv(1024).decode("utf-8")

            if not mensaje:
                print("\n[INFO] El servidor cerró la conexión.")
                break

```

```

        print(f"\n{mensaje}")
        print("> ", end="", flush=True)

    except:
        print("\n[INFO] Se perdió la conexión con el servidor.")
        break

def iniciar_cliente():
    cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    try:
        cliente.connect((HOST, PORT))

        print("=====")
        print("          CLIENTE CHAT")
        print("=====")

        nombre = input("Ingrese su nombre de usuario: ")

        cliente.send(nombre.encode("utf-8"))

        hilo = threading.Thread(
            target=recibir_mensajes,
            args=(cliente,),
            daemon=True
        )

        hilo.start()

        while True:
            mensaje = input("> ")

            cliente.send(mensaje.encode("utf-8"))

            if mensaje == "/quitar":
                break

    except ConnectionRefusedError:
        print("[ERROR] No se pudo conectar con el servidor.")

    except Exception as error:

```

```

        print(f"[ERROR] {error}")

    finally:
        cliente.close()
        print("Cliente desconectado.")

if __name__ == "__main__":
    iniciar_cliente()

```

Pantalla de ejecución e interacción con diferentes clientes:

```

Windows PowerShell
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Leo - ('127.0.0.1', 52078)
[Leo] Hola servidor
[DESCONECTADO] Leo
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Hola - ('127.0.0.1', 54244)
[Hola] Leo
[DESCONECTADO] Hola
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] Leo - ('127.0.0.1', 54249)
[Leo] Holaaa
[CONEXIONES ACTIVAS] 2
[NUEVO CLIENTE] Francisco - ('127.0.0.1', 58336)
[CONEXIONES ACTIVAS] 3
[NUEVO CLIENTE] Leopoldo - ('127.0.0.1', 58339)
[Leopoldo] Hola a todos
[Francisco] Hola Leopoldo, me approves?
[Leo] Hola, yo también me llamo Leo
[DESCONECTADO] Leo
[CONEXIONES ACTIVAS] 3
[NUEVO CLIENTE] Tumacho - ('127.0.0.1', 55169)
[DESCONECTADO] Tumacho
[Leopoldo] Hola chicos
[Leopoldo] Hola profesor
[Francisco] -
[Francisco] sdf
[Francisco] d
[Leopoldo] Hola chicos
[Francisco] Hola profesor
[DESCONECTADO] Francisco
[CONEXIONES ACTIVAS] 2
[NUEVO CLIENTE] Pablo - ('127.0.0.1', 61673)
[DESCONECTADO] Pablo
[CONEXIONES ACTIVAS] 2
[NUEVO CLIENTE] Leonel - ('127.0.0.1', 61328)
[DESCONECTADO] Leopoldo

Windows PowerShell
[Francisco] sdf
>
[Francisco] d
> Hola chicos
>
[Francisco] Hola profesor
>
[SERVIDOR] Francisco se desconectó del chat.
>
[SERVIDOR] Pablo ingresó al chat.
>
[SERVIDOR] Pablo se desconectó del chat.
>
[SERVIDOR] Leonel ingresó al chat.
> /quitar
[INFO] El servidor cerró la conexión.
Cliente desconectado.
PS D:\LICENCIATURA EN SISTEMAS\CUARTO AÑO\Redes de Datos\Trabajos prácticos\
Resolución Tps\TPN3\TPN3.2_Sockets>

Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\LICENCIATURA EN SISTEMAS\CUARTO AÑO\Redes de Datos\Trabajos prácticos\
Resolución Tps\TPN3\TPN3.2_Sockets> py cliente.py
=====
CLIENTE CHAT
=====
Ingrese su nombre de usuario: Leonel

[SERVIDOR] Conexión establecida correctamente.
> > /listar

[SERVIDOR] Usuarios conectados: Leopoldo, Leonel>
>
[SERVIDOR] Leopoldo se desconectó del chat.
> |

```

Ejercicio 2)

1. Desarrollo del nuevo cliente.

Se desarrolló un nuevo cliente utilizando Java, mientras que el servidor del Ejercicio 1 continúa implementado en Python.

El cliente Java solicita al usuario la dirección IP y el puerto del servidor:

```

System.out.println("=====");
System.out.println("      CLIENTE CHAT JAVA");
System.out.println("=====");

System.out.print("Ingrese IP del servidor: ");
String host = teclado.nextLine();

System.out.print("Ingrese puerto del servidor: ");
int puerto;

```

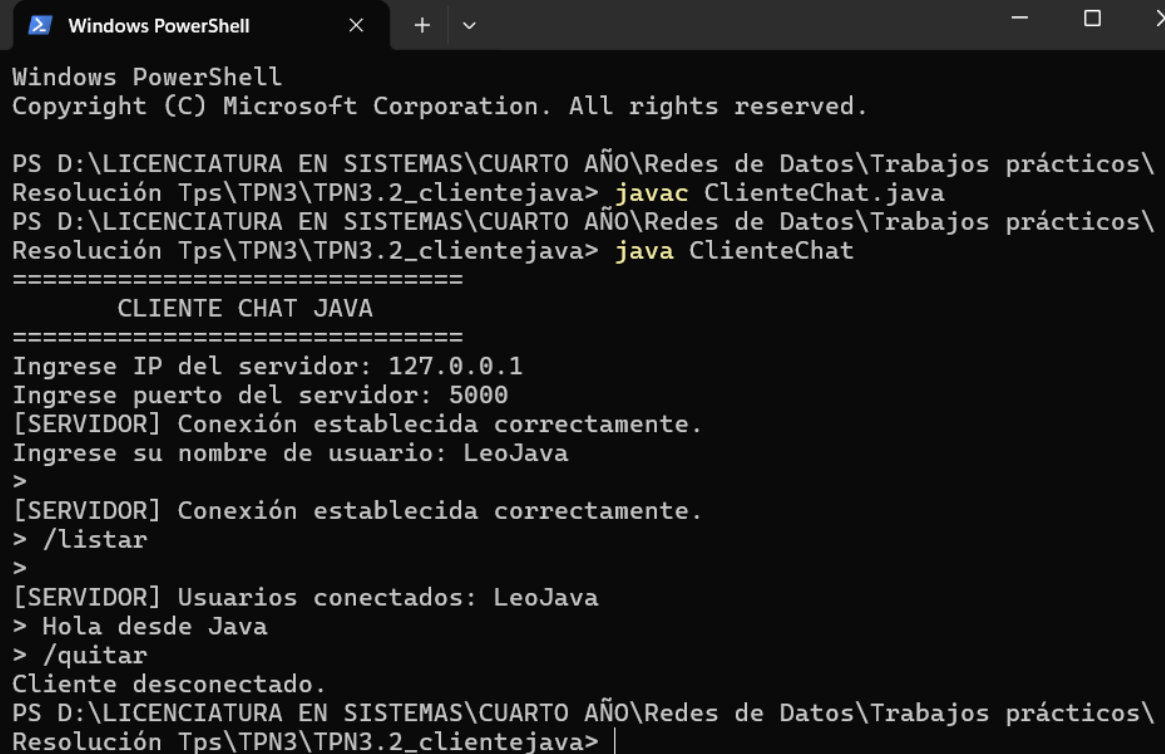
Luego establece la conexión TCP mediante:

```

Socket socket = new Socket(host, puerto);

```

Cliente Java conectado al servidor Python



```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\LICENCIATURA EN SISTEMAS\CUARTO AÑO\Redes de Datos\Trabajos prácticos\
Resolución Tps\TPN3\TPN3.2_clientejava> javac ClienteChat.java
PS D:\LICENCIATURA EN SISTEMAS\CUARTO AÑO\Redes de Datos\Trabajos prácticos\
Resolución Tps\TPN3\TPN3.2_clientejava> java ClienteChat
=====
      CLIENTE CHAT JAVA
=====
Ingrese IP del servidor: 127.0.0.1
Ingrese puerto del servidor: 5000
[SERVIDOR] Conexión establecida correctamente.
Ingrese su nombre de usuario: LeoJava
>
[SERVIDOR] Conexión establecida correctamente.
> /listar
>
[SERVIDOR] Usuarios conectados: LeoJava
> Hola desde Java
> /quitar
Cliente desconectado.
PS D:\LICENCIATURA EN SISTEMAS\CUARTO AÑO\Redes de Datos\Trabajos prácticos\
Resolución Tps\TPN3\TPN3.2_clientejava> |

```

Se observa la conexión exitosa del cliente desarrollado en Java con el servidor implementado previamente en Python.

2. Comunicación entre aplicaciones de distintos lenguajes

Una vez conectado, el cliente Java puede enviar y recibir mensajes mediante el mismo servidor utilizado por los clientes Python.

El cliente Java envía información mediante:

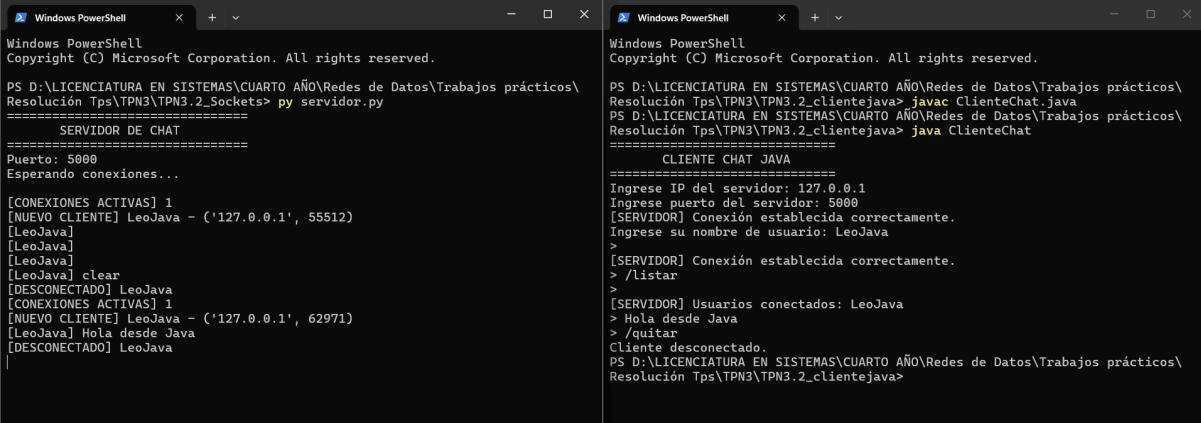
```
salida.println(nombre);
```

y recibe mensajes utilizando:

```
int bytesLeidos = entrada.read(buffer);
```

De esta manera se logró comunicación entre aplicaciones desarrolladas en lenguajes diferentes, utilizando TCP y un formato común de mensajes.

Comunicación Java - Python



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\LICENCIATURA EN SISTEMAS\CUARTO AÑO\Redes de Datos\Trabajos prácticos\
Resolución Tps\TPN3\TPN3.2_Sockets> py servidor.py
=====
SERVIDOR DE CHAT
=====
Puerto: 5000
Esperando conexiones...

[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] LeoJava - ('127.0.0.1', 55512)
[LeoJava]
[LeoJava]
[LeoJava]
[LeoJava] clear
[DESCONECTADO] LeoJava
[CONEXIONES ACTIVAS] 1
[NUEVO CLIENTE] LeoJava - ('127.0.0.1', 62971)
[LeoJava] Hola desde Java
[DESCONECTADO] LeoJava

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\LICENCIATURA EN SISTEMAS\CUARTO AÑO\Redes de Datos\Trabajos prácticos\
Resolución Tps\TPN3\TPN3.2_clientejava> javac ClienteChat.java
PS D:\LICENCIATURA EN SISTEMAS\CUARTO AÑO\Redes de Datos\Trabajos prácticos\
Resolución Tps\TPN3\TPN3.2_clientejava> java ClienteChat
=====
CLIENTE CHAT JAVA
=====
Ingrese IP del servidor: 127.0.0.1
Ingrese puerto del servidor: 5000
[SERVIDOR] Conexión establecida correctamente.
Ingrese su nombre de usuario: LeoJava
>
[SERVIDOR] Conexión establecida correctamente.
> /listar
>
[SERVIDOR] Usuarios conectados: LeoJava
> Hola desde Java
> /quitar
Cliente desconectado.
PS D:\LICENCIATURA EN SISTEMAS\CUARTO AÑO\Redes de Datos\Trabajos prácticos\
Resolución Tps\TPN3\TPN3.2_clientejava>
```

La captura demuestra la interoperabilidad entre el cliente desarrollado en Java y el cliente desarrollado en Python, utilizando el mismo servidor central.

3. Comando “/listar”

El cliente Java mantiene las funcionalidades implementadas anteriormente.

Al escribir “/listar” el servidor devuelve la lista de usuarios conectados.

```
[SERVIDOR] Conexión establecida correctamente.  
> /listar  
>  
[SERVIDOR] Usuarios conectados: LeoJava
```

Se verifica que el cliente Java puede utilizar correctamente el comando `/listar` y recibir la lista de usuarios conectados.

4. Comando “/quitar”

El cliente Java también permite finalizar la conexión mediante: `/quitar`.

El programa envía el comando al servidor y posteriormente cierra el socket.

```
> /quitar  
Cliente desconectado.  
PS D:\LICENCIATURA EN SISTEMAS\CUARTO AÑO\Redes de Datos\Trabajos prácticos\  
Resolución Tps\TPN3\TPN3.2_clientejava>
```

Se observa la desconexión controlada del cliente Java mediante el comando `/quitar`.

Conclusión:

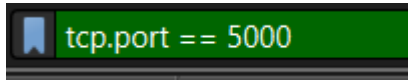
Se desarrolló correctamente un nuevo cliente de chat utilizando Java, capaz de conectarse al servidor desarrollado en Python mediante sockets TCP. El cliente solicita la dirección IP y el puerto del servidor, informa el estado de la conexión y mantiene las funcionalidades de envío y recepción de mensajes, `/listar` y `/quitar`. Además, se comprobó la interoperabilidad entre clientes desarrollados en distintos lenguajes.

Ejercicio 3)

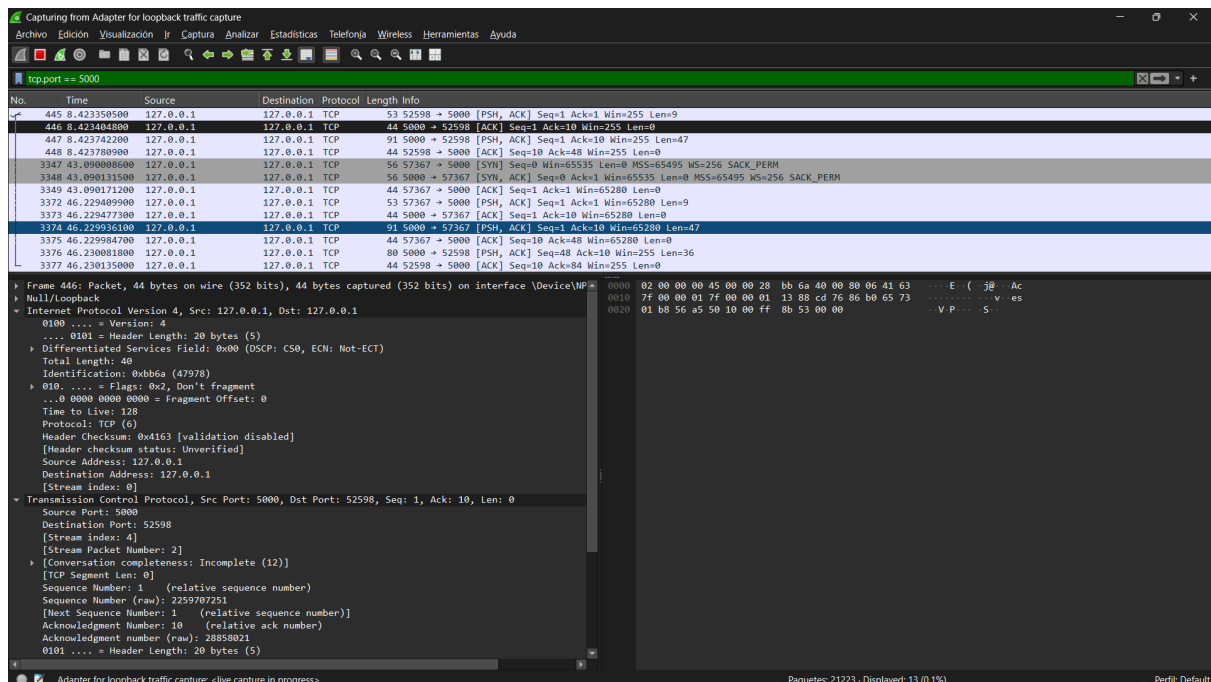
- Identificación de IP y puertos.

Se utilizó Wireshark para capturar el tráfico generado durante las conexiones entre los clientes y el servidor de chat.

Para limitar la captura al tráfico correspondiente a la aplicación se utilizó el filtro:



Durante las pruebas locales, tanto cliente como servidor utilizaron la dirección: 127.0.0.1, correspondiente a la interfaz de loopback.



En la captura se identifican las direcciones IP y los puertos de origen y destino correspondientes a una conexión TCP entre un cliente y el servidor. El servidor utiliza el puerto 5000, mientras que el cliente utiliza un puerto dinámico asignado por el sistema operativo.

- Conexiones de varios clientes

The image shows a Wireshark packet capture from an adapter for loopback traffic. The filter is 'tcp.port == 5000'. The packet list shows a series of TCP connections from various source IP addresses (e.g., 127.0.0.1) to the destination 127.0.0.1 on port 5000. The connections are established using the SYN-ACK sequence. The packet details pane shows the selected packet (No. 445) as an Internet Protocol Version 4 packet with source 127.0.0.1 and destination 127.0.0.1. The transmission control protocol details show a sequence number of 52598, acknowledgment number of 5000, and a window size of 255. The packet bytes pane shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
445	8.423358508	127.0.0.1	127.0.0.1	TCP	53	52598 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=255 Len=9
446	8.423404800	127.0.0.1	127.0.0.1	TCP	44	5000 → 52598 [ACK] Seq=1 Ack=10 Win=255 Len=0
447	8.423742200	127.0.0.1	127.0.0.1	TCP	91	5000 → 52598 [PSH, ACK] Seq=1 Ack=10 Win=255 Len=47
448	8.423780900	127.0.0.1	127.0.0.1	TCP	44	52598 → 5000 [ACK] Seq=10 Ack=48 Win=255 Len=0
3347	43.090006600	127.0.0.1	127.0.0.1	TCP	56	57367 → 5000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
3348	43.090131500	127.0.0.1	127.0.0.1	TCP	56	5000 → 57367 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
3349	43.090171200	127.0.0.1	127.0.0.1	TCP	44	57367 → 5000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
3372	46.229409900	127.0.0.1	127.0.0.1	TCP	53	57367 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=9
3373	46.229477300	127.0.0.1	127.0.0.1	TCP	44	5000 → 57367 [ACK] Seq=1 Ack=10 Win=65280 Len=0
3374	46.229936100	127.0.0.1	127.0.0.1	TCP	91	5000 → 57367 [PSH, ACK] Seq=1 Ack=10 Win=65280 Len=47
3375	46.229984700	127.0.0.1	127.0.0.1	TCP	44	57367 → 5000 [ACK] Seq=10 Ack=48 Win=65280 Len=0
3376	46.230081800	127.0.0.1	127.0.0.1	TCP	80	5000 → 52598 [PSH, ACK] Seq=48 Ack=10 Win=255 Len=36
3377	46.230135000	127.0.0.1	127.0.0.1	TCP	44	52598 → 5000 [ACK] Seq=10 Ack=84 Win=255 Len=0
31226	678.297367200	127.0.0.1	127.0.0.1	TCP	63	57367 → 5000 [PSH, ACK] Seq=10 Ack=48 Win=65280 Len=19
31227	678.297423300	127.0.0.1	127.0.0.1	TCP	44	5000 → 57367 [ACK] Seq=48 Ack=29 Win=65280 Len=0
31228	678.297906600	127.0.0.1	127.0.0.1	TCP	71	5000 → 52598 [PSH, ACK] Seq=84 Ack=10 Win=255 Len=27
31229	678.297955800	127.0.0.1	127.0.0.1	TCP	44	52598 → 5000 [ACK] Seq=10 Ack=111 Win=255 Len=0
31442	688.380011000	127.0.0.1	127.0.0.1	TCP	71	52598 → 5000 [PSH, ACK] Seq=10 Ack=111 Win=255 Len=27
31443	688.380064500	127.0.0.1	127.0.0.1	TCP	44	5000 → 52598 [ACK] Seq=111 Ack=37 Win=255 Len=0
31444	688.380559900	127.0.0.1	127.0.0.1	TCP	83	5000 → 57367 [PSH, ACK] Seq=48 Ack=29 Win=65280 Len=39
31445	688.380612700	127.0.0.1	127.0.0.1	TCP	44	57367 → 5000 [ACK] Seq=29 Ack=87 Win=65280 Len=0

Frame 445: Packet, 53 bytes on wire (424 bits), 53 bytes captured (424 bits) on interface \Device\NPF...
Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 52598, Dst Port: 5000, Seq: 1, Ack: 1, Len: 9
Data (9 bytes)

En la captura se observan distintas conexiones TCP hacia el servidor de chat. Los clientes utilizan puertos de origen diferentes, en este caso 52598 y 57367, mientras que ambos se comunican con el puerto 5000 del servidor. Esto permite identificar de forma independiente cada conexión establecida.

- Visualización de la LU dentro del paquete TCP

The image shows a Wireshark packet capture from an adapter for loopback traffic. The filter is 'tcp.port == 5000'. The packet list shows a single TCP connection from source IP 127.0.0.1 to destination 127.0.0.1 on port 5000. The connections are established using the SYN-ACK sequence. The packet details pane shows the selected packet (No. 47142) as an Internet Protocol Version 4 packet with source 127.0.0.1 and destination 127.0.0.1. The transmission control protocol details show a sequence number of 52598, acknowledgment number of 5000, and a window size of 18. The packet bytes pane shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
445	8.423358508	127.0.0.1	127.0.0.1	TCP	53	52598 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=255 Len=9
446	8.423404800	127.0.0.1	127.0.0.1	TCP	44	5000 → 52598 [ACK] Seq=1 Ack=10 Win=255 Len=0
447	8.423742200	127.0.0.1	127.0.0.1	TCP	91	5000 → 52598 [PSH, ACK] Seq=1 Ack=10 Win=255 Len=47
448	8.423780900	127.0.0.1	127.0.0.1	TCP	44	52598 → 5000 [ACK] Seq=10 Ack=48 Win=255 Len=0
3347	43.090006600	127.0.0.1	127.0.0.1	TCP	56	57367 → 5000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
3348	43.090131500	127.0.0.1	127.0.0.1	TCP	56	5000 → 57367 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
3349	43.090171200	127.0.0.1	127.0.0.1	TCP	44	57367 → 5000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
3372	46.229409900	127.0.0.1	127.0.0.1	TCP	53	57367 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=9
3373	46.229477300	127.0.0.1	127.0.0.1	TCP	44	5000 → 57367 [ACK] Seq=1 Ack=10 Win=65280 Len=0
3374	46.229936100	127.0.0.1	127.0.0.1	TCP	91	5000 → 57367 [PSH, ACK] Seq=1 Ack=10 Win=65280 Len=47
3375	46.229984700	127.0.0.1	127.0.0.1	TCP	44	57367 → 5000 [ACK] Seq=10 Ack=48 Win=65280 Len=0
3376	46.230081800	127.0.0.1	127.0.0.1	TCP	80	5000 → 52598 [PSH, ACK] Seq=48 Ack=10 Win=255 Len=36
3377	46.230135000	127.0.0.1	127.0.0.1	TCP	44	52598 → 5000 [ACK] Seq=10 Ack=84 Win=255 Len=0
31226	678.297367200	127.0.0.1	127.0.0.1	TCP	63	57367 → 5000 [PSH, ACK] Seq=10 Ack=48 Win=65280 Len=19
31227	678.297423300	127.0.0.1	127.0.0.1	TCP	44	5000 → 57367 [ACK] Seq=48 Ack=29 Win=65280 Len=0
31228	678.297906600	127.0.0.1	127.0.0.1	TCP	71	5000 → 52598 [PSH, ACK] Seq=84 Ack=10 Win=255 Len=27
31229	678.297955800	127.0.0.1	127.0.0.1	TCP	44	52598 → 5000 [ACK] Seq=10 Ack=111 Win=255 Len=0
31442	688.380011000	127.0.0.1	127.0.0.1	TCP	71	52598 → 5000 [PSH, ACK] Seq=10 Ack=111 Win=255 Len=27
31443	688.380064500	127.0.0.1	127.0.0.1	TCP	44	5000 → 52598 [ACK] Seq=111 Ack=37 Win=255 Len=0
31444	688.380559900	127.0.0.1	127.0.0.1	TCP	83	5000 → 57367 [PSH, ACK] Seq=48 Ack=29 Win=65280 Len=39
31445	688.380612700	127.0.0.1	127.0.0.1	TCP	44	57367 → 5000 [ACK] Seq=29 Ack=87 Win=65280 Len=0

Frame 47142: Packet, 62 bytes on wire (496 bits), 62 bytes captured (496 bits) on interface \Device\NPF...
Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 5000, Dst Port: 52598, Seq: 111, Ack: 37, Len: 18
Data (18 bytes)
Data: 5b4c656f4a6176615d204c555f3630303839
[Length: 18]

En la captura se observa el mensaje enviado por el cliente dentro de los datos del paquete TCP. Al no utilizarse ningún mecanismo de cifrado en la aplicación desarrollada, el contenido puede visualizarse directamente mediante Wireshark.

CONCLUSIÓN:

Mediante Wireshark se verificó el funcionamiento de las conexiones TCP establecidas por los clientes hacia el servidor. Se identificaron las direcciones IP, los puertos de origen y destino y las tuplas correspondientes a cada conexión. También se comprobó que los mensajes enviados por el chat viajan sin cifrado, pudiendo visualizarse su contenido directamente durante la captura del tráfico.

EN CASO QUE SE REQUIERA REVISIÓN DEL CÓDIGO, DEJO EL REPOSITORIO DE GITHUB PARA VERIFICARLO:

https://github.com/Leo-alegre9/TPN3.2_Programacion_de_sockets